

Experiment 10: Map Coloring using CSP

Aim

To implement the Map Coloring problem using Constraint Satisfaction Problem (CSP).

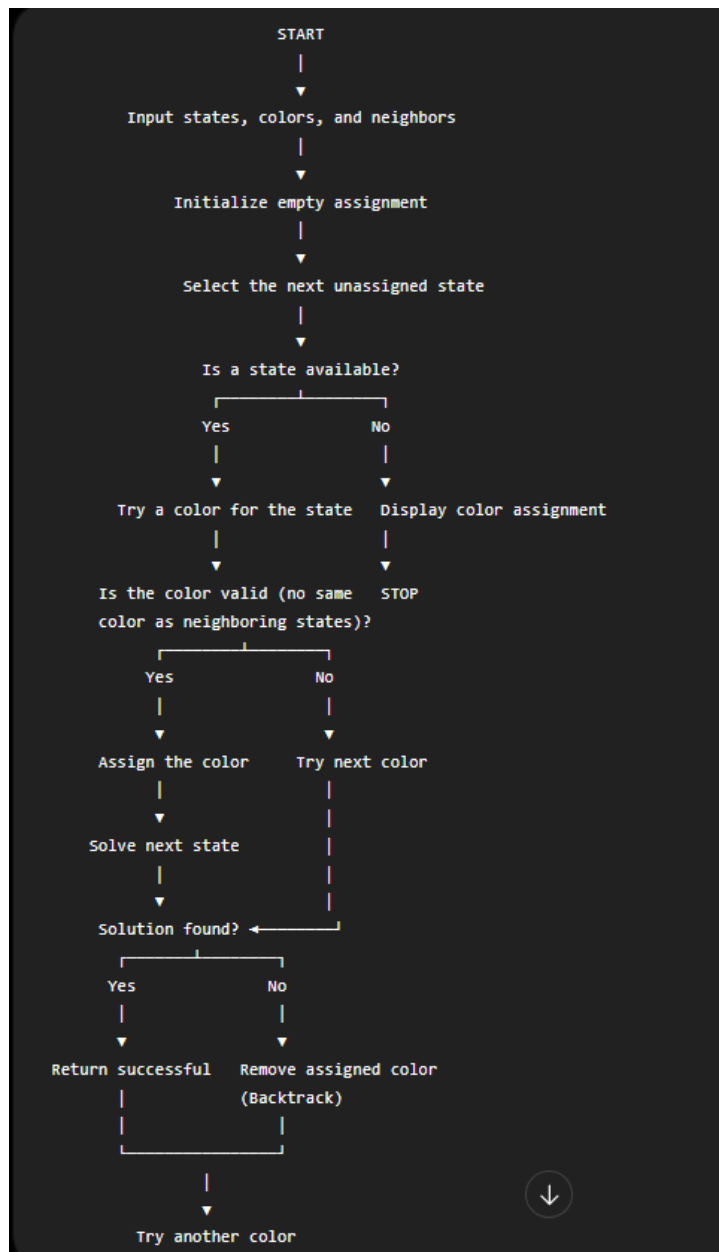
Objective

- To assign colors to regions.
- To ensure adjacent regions have different colors.

Algorithm

1. Create the map and adjacency list.
2. Define available colors.
3. Assign colors recursively.
4. Check whether neighboring regions have different colors.
5. If valid, continue.
6. Display the final coloring.

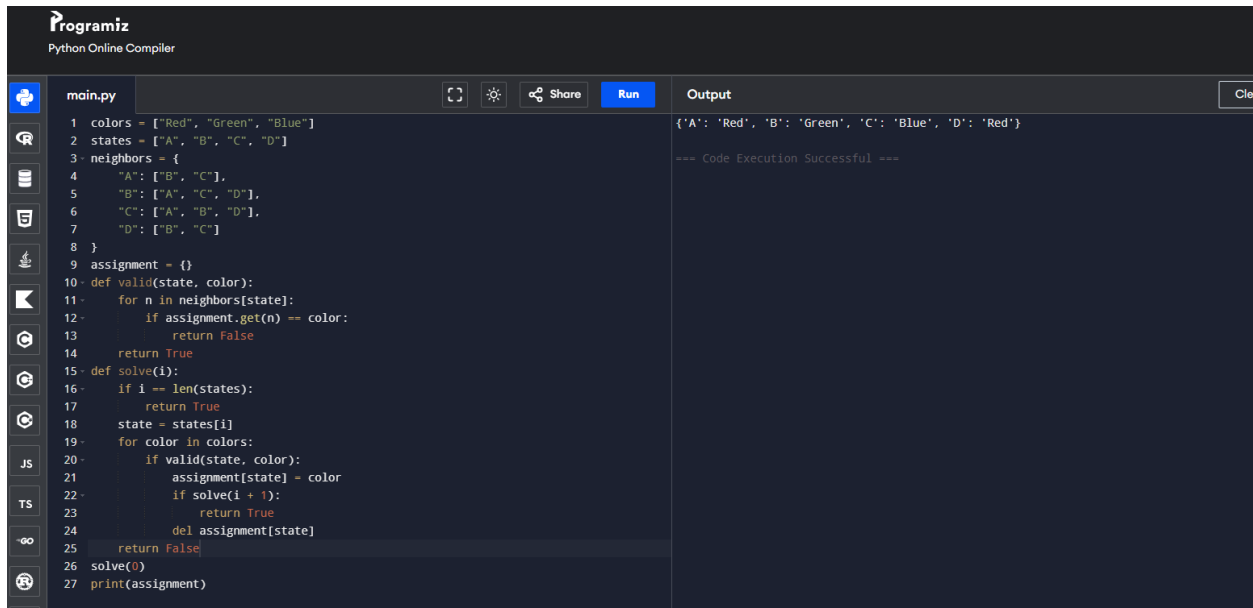
Flowchart



Program

```
colors = ["Red", "Green", "Blue"]
states = ["A", "B", "C", "D"]
neighbors = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}
assignment = {}
def valid(state, color):
    for n in neighbors[state]:
        if assignment.get(n) == color:
            return False
    return True
def solve(i):
    if i == len(states):
        return True
    state = states[i]
    for color in colors:
        if valid(state, color):
            assignment[state] = color
            if solve(i + 1):
                return True
            del assignment[state]
    return False
solve(0)
print(assignment)
```

Output



The screenshot shows the Programiz Python Online Compiler interface. On the left, a sidebar contains icons for file management and a language selector with options for JS, TS, and Python (selected). The main editor displays a Python script named 'main.py' with the following code:

```
1 colors = ["Red", "Green", "Blue"]
2 states = ["A", "B", "C", "D"]
3 neighbors = {
4     "A": ["B", "C"],
5     "B": ["A", "C", "D"],
6     "C": ["A", "B", "D"],
7     "D": ["B", "C"]
8 }
9 assignment = {}
10 def valid(state, color):
11     for n in neighbors[state]:
12         if assignment.get(n) == color:
13             return False
14     return True
15 def solve(i):
16     if i == len(states):
17         return True
18     state = states[i]
19     for color in colors:
20         if valid(state, color):
21             assignment[state] = color
22             if solve(i + 1):
23                 return True
24     del assignment[state]
25     return False
26 solve(0)
27 print(assignment)
```

On the right, the 'Output' panel shows the result of the execution:

```
{'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}

=== Code Execution Successful ===
```

Result

The map was successfully colored without violating any constraints.

Conclusion

The CSP approach successfully assigns colors so that no two adjacent regions share the same color.